

04.AI 学习之路之「Skill：把好用的 Prompt 变成一键技能」

来自： 帅张和他的朋友们

 stormzhang

2026年05月13日 17:21



扫码
查看更

这是 AI 学习之路系列的第五篇。

上一篇讲了全局配置——用 CLAUDE.md 让 AI 永远记住你是谁、怎么沟通、哪些事不能干。

配置解决的是「AI 的基础人设」。但用一段时间你会撞上另一个问题：**同样的场景，你每次都在重新写 Prompt。**

每次让 AI 润色邮件，都得重新交代：「语气正式但不生硬，保留核心意思，控制在 200 字以内……」。每次让它帮你总结长文，又得从头说一遍。

配置解决了「AI 不认识你」，今天解决「AI 不记得你常做的事」。方法就是——Skill。

1 什么是 Skill

先说结论：**Skill 就是一套针对具体场景写好的 Prompt 指令，保存下来反复用。**

本质上，skill 就是一份 markdown 文档。

类比：手机里的快捷指令。你设置好「早安模式」，每天点一下，手机自动关闹钟、打开日历、调亮度——不用每次手动操作三次。Skill 也一样，把一个场景的完整 Prompt 写好存下来，以后一句话触发，不用从头写。

跟每次现写的 Prompt 比，区别就一句话：**Prompt 解决当下，Skill 解决重复。**

你今天临时要翻译一段英文，随手写个 Prompt 就够了，一次性的事，没必要存。但你**每周都要整理会议纪要**，每次都得重新交代格式要求——这就该做成 Skill。写一次，以后直接用，格式一致，质量稳定。

什么时候该做 Skill？三个信号：

同一个 Prompt 你已经写了三遍以上——场景够高频，值得固化

每次写出来效果不一样——需要把「怎么写效果好」锁定下来

你的 Prompt 超过 5 行——指令够复杂，每次现写容易漏

我自己最早意识到该做 Skill，是因为写公众号。每次让 AI 帮我改稿，都要花两分钟重新交代风格要求——口语化、不要书面腔、段落要短、不要用「赋能」「闭环」。第三次写同样的东西时我就烦了：这不就该存下来吗？存下来之后，改稿从「写两分钟指令 + 等结果」变成了「粘贴 Skill + 丢草稿」，每次省两分钟，一周省半小时。

当然这里只提到写作风格，但其实公众号写作是一整套 workflow，先按照自己的风格写作，确认内容 ok 再利用脚本排版，自动生成我要的头图风格，然后一键推送到草稿箱，这种涉及到工作流的 skill，我们后面再说。

2 一个好 Skill 长什么样

先看一个完整例子——「会议纪要整理」Skill：

```
## 角色
你是一位专业的会议纪要整理助手。

## 任务
我会提供会议录音转写稿或手写笔记，你整理成结构化的会议纪要。

## 输出格式
- **会议主题**：
- **参与人员**：
- **核心决议**（每条不超过 20 字）：
```

```
- **待办事项** (含负责人和截止日期):  
- **遗留问题**:  
  
## 约束  
- 用第三人称, 客观陈述  
- 只提取原文里有的信息, 不补充不推测  
- 信息不完整的字段标注「待补充」  
- 待办没提到截止日期的标注「未定」
```

把以上内容保存到一个叫做 meeting_report.md 的文件里，这就是一份最小可用的 skill。

看出规律了吗？一个 Skill 本质就是一份写给 AI 的角色说明书，包含四块：

模块	作用	一句话说明
角色	让 AI 知道自己是谁	你扮演什么专家
任务	干什么，输入是什么	我给你什么，你帮我做什么
输出格式	结果长什么样	用什么结构呈现
约束	边界在哪	哪些事不能干

不是每块都必须有--角色 + 任务 + 输出格式是基本盘，约束看场景加。

你可能觉得这跟第 02 篇讲的万能公式很像。没错，Skill 就是万能公式的固化版——区别在于 Prompt 是临时写的，Skill 是经过测试和迭代、确认效果好了才保存下来的。

3 从零做一个 Skill

第一步：锁定场景

好 Skill 的第一条标准：**场景足够具体**。

- ✔ 「每周整理会议纪要」
- ✔ 「把长文章总结成 3 条要点」
- ✔ 「把我的草稿改成公众号风格」
- ✘ 「万能写作助手」
- ✘ 「AI 全能秘书」

越具体越好用。「万能」听起来厉害，实际上万能等于万不能--什么都能做一点，什么都做不精。

我自己踩过这个坑。最早做了个「写作全能助手」Skill，我想的是公众号写作，星球教程，工作周报之类的，都用一个 skill 来搞定，既能改稿又能写标题还能做排版建议，结果哪个都做不好——因为你没法在一个 Skill 里把三种完全不同的任务都交代清楚。后来拆成三个独立 Skill，每个只做一件事，效果立刻好了。

所以，这里大家记得，每个 skill 只解决单一场景，越具体的场景，意味着需求越明确，AI 才更能针对性的去解决。

第二步：写初版

用四块结构（角色 + 任务 + 输出格式 + 约束），把你脑子里的要求写出来。

一个关键原则：**把你以前手动写 Prompt 时反复强调的那些重复要求，全部固化进去。**

比如你每次让 AI 写周报都要补一句「用数据说话，不要写空洞的定性描述」——那就直接写进约束里，以后不用每次提醒。不用追求完美，先把想到的写上，后面会迭代。

第三步：测试迭代

把 Skill 内容粘贴到 AI 对话开头，丢一份**真实素材**进去（不要用假数据，真实场景才能暴露问题），看结果。
重点检查三件事：

格式对不对？



有没有漏信息，或者加了你不想要的东西？

语气风格符合预期吗？

不满意就改 Skill 文本。几个常见调整方向：

结果太长 → 加字数限制：「每条不超过 X 字」

风格不对 → 在约束里加排除项：「不要用书面语」「不要加 emoji」

漏关键信息 → 在输出格式里补上对应字段

AI 自由发挥太多 → 加「只提取原文信息，不补充不推测」

通常 2-3 轮就能调到满意。这个迭代过程很重要——一个没经过测试的 Skill 和一个临时写的 Prompt 没有本质区别。

说句实话，我自己做的每个 Skill 都至少改过两轮。第一版写出来觉得挺完整了，实际一跑就会发现问题。比如我的「长文总结」Skill，第一版只写了「总结成 3 条要点」，结果 AI 每次给的要点都是原文前三段的内容，后面的重点全丢了。加了一条「要点必须覆盖全文，不能集中在某一段」才解决。这种问题你不跑真实素材根本发现不了。

第四步：保存复用

Skill 定型之后，找个地方存起来。随着专属于你自己的 skill 文档越来越多，你可以新建一个专属于你自己的 my-skills 项目，把你自己一些特定场景总结的 skill markdown 文件全扔进去就好，然后交给 Claude Code 或者 Codex，让他帮你自己定义什么时候调用什么 skill。

针对你用 Claude Code 或者 Codex，有更高效的方式——后面第 5 节专门讲。

4 进阶：让 Skill 更强的 5 个技巧

上面讲的是「怎么做一个能用的 Skill」，这部分讲怎么让它**更好用**。

技巧一：在 Skill 里嵌示例

第 02 篇讲过「给示例比说十遍管用」，这个技巧在 Skill 里用起来效果更明显。

比如你做了一个「周报生成」Skill，在约束里写了「语言简练、突出成果、带数据」，但 AI 每次写出来的风格还是不对味。与其继续加约束，不如直接给它看一段你满意的周报：

示例

输入：「本周完成了用户调研，访谈了 8 个客户，整理了竞品分析报告」

输出：「**本周成果**

- 完成 8 场客户深度访谈，产出 3 个核心需求洞察
- 输出竞品分析报告（覆盖 5 家），识别 2 个差异化切入点

下周计划

- 基于调研结论输出需求优先级排序（周三前）
- 与技术团队对齐方案可行性（周五前）」

放 1-2 组「输入 → 输出」示例就够了。AI 会从示例里自动学会你要的格式、详略程度和用词风格——比你用文字描述精准得多。

什么时候用： 你发现不管怎么调约束，AI 的输出风格总是不对味的时候，大概率是该给示例了。

技巧二：加自检步骤

Skill 执行完，AI 就直接把结果甩给你了。但你有没有遇到过：拿到结果一看，超字数了、出现了禁用词、格式不对——又得让它重改一遍。

解决办法：在 Skill 末尾加一段自检清单，让 AI 输出后自己先过一遍。

自检（输出前必须逐条检查）

- [] 每条要点是否控制在 30 字以内？超过的压缩
- [] 是否出现了「赋能」「闭环」等禁用词？有就替换
- [] 总字数是否超过 500 字？超过的精简
- [] 格式是否和示例一致？



我最开始做 Skill 没加自检，结果每次拿到输出都要自己数字数、肉眼扫禁用词，本质上还是人工质检。加了自检之后，AI 交付的结果基本能直接用，偶尔有问题它自己就改了，省了很多来回。

什么时候用： 你的 Skill 有明确的硬性约束（字数、格式、禁用词）时，加自检最有效。

技巧三：动态变量

一个邮件润色 Skill，你有时候需要正式语气，有时候需要轻松语气，有时候是道歉场景。做三个 Skill? 太笨了。
更好的做法：在 Skill 里留变量槽位，每次只改一个参数。

```
## 任务
帮我润色以下邮件草稿。

## 参数（每次使用时指定）
- 语气：[正式 / 轻松 / 道歉 / 催促]
- 收件人关系：[上级 / 同事 / 客户 / 供应商]
- 字数上限：[不限 / 200字 / 500字]

## 约束
- 根据「语气」和「收件人关系」调整措辞
- 保留原文核心意思，不增加原文没有的信息
```

用的时候在对话里说：「语气：道歉，收件人：客户，字数：200 字以内」，然后粘贴草稿。一个 Skill 覆盖四五种场景。

什么时候用： 你发现同一类任务经常需要微调某几个参数时，与其做多个 Skill，不如做一个带变量的。

技巧四：Skill 组合链

单个 Skill 解决单个场景。但真实工作中，很多任务是多步的——光做完第一步不够，还得接着处理。
比如你收到一篇 5000 字的行业报告，最终目的是发一条朋友圈。直接让 AI「把这篇报告改成朋友圈文案」，结果大概率不好——因为从 5000 字到 100 字，跨度太大，AI 不知道该保留什么。
更好的做法：用两个 Skill 串联。

```
第一步：用「长文总结 Skill」
→ 输入 5000 字报告，输出 3 条核心要点

第二步：用「社交文案 Skill」
→ 输入 3 条要点，输出一条朋友圈文案
```

每步的 Skill 都很专注，各做好自己的事，串起来就能处理复杂任务。

这个思路其实就是第 02 篇讲过的 Prompt 链——区别在于，Prompt 链的每一步你现写，Skill 组合链的每一步你直接调用已有的 Skill，更快更稳定。

之所以前面特意强调，每个 skill 要尽量场景单一，需求明确，把所有专属你的 skill 统一放到一个项目目录了，正是因为 skill 组合链的使用。
什么时候用： 你发现一个 Skill 很难一步到位出好结果的时候，大概率是任务该拆成两步了。

技巧五：让 AI 帮你写 Skill

第 02 篇讲过一个技巧叫「元提示」——不会写 Prompt? 让 AI 帮你写。这招用在 Skill 上更合适，因为 Skill 比普通 Prompt 结构更复杂，从零写起来有点费劲。

直接把你的需求描述给 AI，让它帮你生成 Skill 初版：

```
「我每周都要整理周报，需求是这样的：把我的工作日志整理成结构化周报，突出成果和数据，附带下周计划。请帮我写一个完整的 Skill，包含角色、任务、输出格式和约束。」
```

AI 生成的初版通常结构比你自己的完整——它不会忘记加约束，不会漏输出格式。你拿到之后再根据自己的实际需求调整细节就行。

我自己现在做新 Skill 都是这个流程：**先让 AI 出初版 → 我来删减和调整 → 测试迭代**。比从空白文档开始写快很多。特别是约束部分，AI 经常能帮你想到你自己漏掉的边界条件——比如「如果原文没提到截止日期怎么办」「如果内容超过字数限制怎么处理」。

什么时候用： 你知道自己想要什么，但不知道怎么结构化地写出来的时候。让 AI 打底，你来调，效率最高。



这条特别好用，尤其在遇到你不懂的领域，比如你不懂视频分镜概念，不懂音乐专业名词，这时候直接问 AI，告诉你的需求和你想做的事，越具体越好，然后让 AI 帮你生成专属 skill，然后不断给 AI 施压提问，反复优化。

5 在 Claude Code 和 Codex 里用 Skill

前面讲的都是通用方法——不管你用什么 AI 工具都适用。但如果你用的是 Claude Code 或 Codex，它们有**原生的 Skill 支持**，体验好得多。

Claude Code：斜杠命令

Claude Code 支持把 Skill 做成**斜杠命令**——你在对话框里输入 `/命令名`，它自动加载对应的 Skill，不用复制粘贴任何东西。

还记得我的一个 AI Translate 开源项目呢，本质就是定义了翻译的 skill，然后自定义了命令，在对话框输入 `/t`，直接加载我定义的翻译 skill。

怎么做：

第一步，创建命令文件。Claude Code 的斜杠命令就是一个 Markdown 文件，放在固定目录下：

层级	目录位置	作用范围
全局	<code>~/.claude/skills/</code>	所有项目通用
项目	项目目录 <code>/.claude/skills/</code>	仅当前项目

全局命令适合通用场景：比如翻译，设计模板之类的——不管你在哪个项目下都能用。项目命令适合专属场景：比如某个项目的部署脚本、特定格式的日报模板。

第二步，写一个 `SKILL.md` 文件，内容就是你的 Skill。最外层设置一个文件名，**文件夹名就是命令名**。

比如你做了一个邮件润色 Skill，保存为 `~/.claude/skills/email/SKILL.md`，以后在 Claude Code 里输入 `/email` 就能用，这里注意下，你定义的 `~/.claude/skills` 后面的文件夹名，就是你在对话框对应的 `/命令`。

第三步，在 Claude Code 里输入 `/email`，Skill 自动加载，直接丢内容进去就行。

动态输入怎么办？

上面的 `/email` Skill，语气是写死的「正式但不生硬」。但你发邮件不可能永远一个语气——给老板汇报要正式，给同事说事要轻松，道歉邮件又是另一个调子。

在 Skill 里加一个 `$ARGUMENTS` 占位符就行：

```
## 任务
帮我润色以下邮件草稿。

## 语气要求
$ARGUMENTS

## 约束
- 保留原文核心意思，不加原文没有的信息
- 控制在 200 字以内
```

用的时候，`/email` 后面粘贴邮件草稿，再跟上你要的语气：

```
/email 邮件内容 诚恳道歉
```

AI 收到的就是：Skill 里的润色指令 + 「诚恳道歉」替换掉 `$ARGUMENTS` + 你粘贴的草稿。

一个 Skill 覆盖各种语气场景，不用重复造轮子。

不过以上只是自定义的简便使用，只是方便大家了解，但是实际使用中，你通过自然语言也是一样，比如上面那条命令，等同于：



「用 email skill 润色下这封邮件内容，然后语气是诚恳道歉。」

我自己的用法：

我用 Claude Code 做了不少斜杠命令。举个最重的例子：我做了一个 `/aidaily` 命令，每天输入一下，自动抓取各大 AI 渠道信息、用 AI 分析每个项目的定位和使用场景、生成结构化 json 数据、刷新 AI Daily News 信息列表、部署到服务器。一个斜杠命令，五六步操作全自动跑完。

这就是 Skill 的终极形态——不只是保存一段 Prompt，而是把一整套工作流打包成一个命令。当然，不用一上来就做这么复杂的。先从简单的开始，一个润色邮件的斜杠命令、一个总结文章的斜杠命令，用起来就已经很爽了。

Codex

Codex 的思路大差不差。只不过它默认的不是斜杠命令，而是 `$`，用法和 Claude Code 几乎差不多，而且 codex 的目录是 `~/codex/skills/email/SKILL.md`。

用法几乎一样，如果有不一样的，直接问 codex 就好。

如果你用其他 AI 工具，其实也一样，最多是存放目录以及支不支持自定义命令的区别，直接问你的 AI 工具就好。



6 动手：做一个你的写作风格 Skill

光看不练假把式。下面用一个真实场景走完全流程——做一个专属于你的写作风格 Skill。

为什么选这个场景

只要你在写公众号、小红书、或者任何需要个人风格的内容，就一定遇到过这个问题：AI 写的东西总有一股「AI 味」——客气、平淡、面面俱到、没个性。你让它帮你改稿，改完觉得「对，但不像我写的」。

问题出在哪？你没告诉 AI 你的风格是什么。做一个写作风格 Skill，把你的风格固化下来，以后 AI 帮你改稿、扩写、润色，出来的就是你的味道。

第一步：回答四个问题

花 5 分钟想清楚这些（真的写下来，不要只在脑子里过）：

1. 你的文章口吻是什么？ 口语化 / 正式 / 犀利 / 温和 / 自嘲.....
2. 你的读者是谁？ 职场人 / 学生 / 创业者 / 宝妈.....
3. 你有什么表达习惯？ 用「你」还是「大家」？ 短句还是长句？ 爱用类比还是数据？
4. 哪些词你绝对不想出现？ 「赋能」「闭环」「在这个快节奏的时代」「希望对你有帮助」.....

第二步：套进模板

角色

你是我的写作助手，负责把我的草稿改写成符合我个人风格的文章。

我的读者

【你的读者画像，例：25–35 岁互联网职场人，关注效率和个人成长】

我的风格

- 【口吻，例：口语化，用「你」「我」，像跟朋友聊天】
- 【节奏，例：多用短句，段落不超过 4 行】
- 【态度，例：有观点，不模棱两可，敢下结论】
- 【特色，例：喜欢用生活类比解释复杂概念】

绝对不用的表达

- 【列出来，例：「赋能」「闭环」「颗粒度」「在这个快节奏的时代」】

示例

以下是一段我满意的文章片段，请学习这个风格：

「[粘贴你写过的一段文字，200 字左右就够]」

```
## 任务
我给你一段草稿，你按以上风格改写。保留核心观点，风格全权交给你。

## 约束
- 删掉了原文某个要点，改完后注明「删除了：xxx」
- 不改变我的核心立场
- 原文逻辑有问题，改完后单独指出，不要偷偷改

## 自检
- [ ] 是否出现了「绝对不用」列表里的词？
- [ ] 段落是否超过 4 行？
- [ ] 风格是否和示例一致？
```

注意这个模板用上了前面讲的两个进阶技巧：**嵌示例**（示例模块）和**自检**（自检模块）。

第三步：测试和迭代

把 Skill 粘贴到 Claude Code 或 Codex 对话开头，丢一段你最近写的草稿进去，看改写结果。
检查三件事：

- 风格对不对？** 是不是你的「味道」？
- 有没有加戏？** 加了你不想到的内容？
- 观点保留了吗？** 核心意思有没有被改掉？

根据结果调整 Skill，常见问题和对应修改：

问题	在 Skill 里加什么
改写太正式	「不要用此外、综上所述等书面过渡词」
段落太长	「每段不超过 3 行，超过了主动断段」
总是加 emoji	「不要添加 emoji，除非原文有」
喜欢加总结段	「结尾不要加总结性段落」

调 2-3 轮，定型。

第四步：保存

其实上面定了之后，这一步你完全可以交给你的 AI，说让他帮你保存这份 skill，作为全局或者当前项目调用，如果你还想使用方便些，那就自定义一个你的专属斜杠指令。

7 不要安装大量的 skill

大家应该都知道，市面上一大堆各种貌似好用的 skills，很多新手最常犯的错误就是上来装一大堆 skills，看到别人推荐就装，这里提醒下，不要这样做，你装的 skills 越多，会导致你的 AI 工具负担越重，消耗 token 也越大，尤其是全局范围内。

最好的做法是，系统全局只装一些官方常用的，特定项目开发，再去搜索安装适合的 skill，而且只装在当前项目。

这里有一个我亲身实践下来最好用的方式：

比如你现在要新开发一个文生图的项目，那么你的流程应该是这样的：

- 问你的 AI，去 GitHub 搜索文生图相关的高评价，质量高，比较前沿的 skill 或者相关开源项目；
- 不管他给你什么结果，都问它，市面上同类项目做一个详细的对比，以表格形式告诉我
- 全面了解之后，再选择一个你认为最适合你的 skill，然后让 AI 安装在当前项目
- 安装完之后，不用去读详细的项目文档，直接问你的 AI，这个 skill 该怎么用，支持什么功能

然后你就可以直接接着按照你的项目去开发了。



8 小结

这篇核心就一件事：把反复使用的 Prompt 固化成可复用的 Skill。

基础流程：锁定场景 → 四块结构写初版（角色 + 任务 + 输出格式 + 约束） → 测试迭代 → 保存复用。

进阶技巧：嵌示例让 AI 精准理解你的风格，加自检减少人工检查，用动态变量一个 Skill 覆盖多种场景，用 Skill 组合链处理复杂任务，不会写就让 AI 帮你打底。

工具落地：Claude Code 用斜杠命令（/命名），Codex 用 \$。

判断该不该做 Skill 的标准很简单：同一个 Prompt 写了三遍以上，就该做成 Skill。

到这里，你已经掌握了跟 AI 协作的三个核心能力：怎么说话（Prompt）、怎么让它记住你（配置）、怎么把好用的指令存下来反复用（Skill）。这些都是单点能力。

看完之后，今天就做一件事：翻翻你最近跟 AI 的对话记录，找一个你这周重复写过的 Prompt，花 10 分钟把它做成 Skill。不用完美，先用起来，后面再迭代。

下一篇讲怎么把这些单点串起来，建立一套你自己的 AI 工作流——不只是用好单个对话，而是把 AI 真正融入你每天的工作节奏。

本系列持续更新，建议收藏。

